

 roshan5038 / EXP---8-THRESHOLDING ▾



 Code  Issues  Pull requests  Actions  Projects  Wiki  Security and quality  Insights

 Settings

 Watch 0 ▾

 Fork 0



☆ 0 stars  0 forks  0 watching  1 branch  0 tags  Activity

 Public repository

 ▾  1 Branch  0 Tags  

 Go to file  T

Go to file

Add file ➕

Code

⋮

 roshan5038 Update README.md

8abfcd3 · 1 minute ago 

 README.md Update README.md 1 minute ago

README



EX 8 Thresholding Techniques Using OpenCV

Aim

To write a Python program using OpenCV to perform image thresholding techniques and compare the results of **Global Thresholding, Adaptive Thresholding, and Otsu's Thresholding**.

Experiment Details

- **Experiment No.:** 8
- **Experiment Name:** Thresholding
- **Name:** Roshan V
- **Register No.:** 212225230232

Learning Objective

The objectives of this experiment are:

- To understand image thresholding.
- To convert a color image into grayscale.

- To perform Global Thresholding.
- To perform Adaptive Thresholding.
- To perform Otsu's Thresholding.
- To compare different thresholding techniques.

Software Used

- Anaconda – Python 3.7
- Jupyter Notebook / VS Code
- OpenCV (cv2)
- NumPy
- Matplotlib

Libraries Used

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```



Algorithm

Step 1: Import Required Libraries

Import OpenCV, NumPy, and Matplotlib.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```



Step 2: Read the Image and Convert to Grayscale

The input image `chip.png` is read using OpenCV and converted from BGR to grayscale.

```
image = cv2.imread('chip.png')
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```



The grayscale image is used as the input for the thresholding operations.

Step 3: Display the Original Image

The original image is displayed using Matplotlib.

```
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
```



Output – Original Image

Original Image



Step 4: Global Thresholding

Global thresholding is applied using a fixed threshold value of 127.

```
_, global_thresholded = cv2.threshold(
    gray_image,
    127,
    255,
    cv2.THRESH_BINARY
)
```



The threshold value used is:

127

Pixels are classified into two groups based on this fixed threshold.

Output – Global Thresholding

Global Thresholding



Step 5: Adaptive Thresholding

Adaptive thresholding is applied using the **Gaussian method**.

```
adaptive_thresholderd = cv2.adaptiveThreshold(  
    gray_image,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11,  
    2  
)
```



The parameters used in the notebook are:

Parameter	Value
Maximum value	255
Adaptive method	ADAPTIVE_THRESH_GAUSSIAN_C
Threshold type	THRESH_BINARY
Block size	11
Constant C	2

Adaptive thresholding calculates the threshold based on local image regions.

Output – Adaptive Thresholding

Adaptive Thresholding



Step 6: Otsu's Thresholding

Otsu's method is applied to automatically determine an appropriate threshold value.

```
_, otsu_thresholderd = cv2.threshold(  
    gray_image,  
    0,  
    255,
```



```
cv2.THRESH_BINARY + cv2.THRESH_OTSU
)
```

The threshold value is set to 0 because Otsu's method automatically determines the optimal threshold from the image.

Output – Otsu's Method



Step 7: Display All Results

The notebook displays the original image and all three thresholding results in a 2 × 2 subplot.

```
# Global Thresholding
plt.subplot(2, 2, 2)
plt.imshow(global_thresholded, cmap='gray')
plt.title("Global Thresholding")
plt.axis('off')

# Adaptive Thresholding
plt.subplot(2, 2, 3)
plt.imshow(adaptive_thresholded, cmap='gray')
plt.title("Adaptive Thresholding")
plt.axis('off')

# Otsu's Method
plt.subplot(2, 2, 4)
plt.imshow(otsu_thresholded, cmap='gray')
plt.title("Otsu's Method")
plt.axis('off')

plt.tight_layout()
plt.show()
```



Complete Program

```
import cv2
import numpy as np
```



```
import matplotlib.pyplot as plt

# Step 2: Read the image and convert to grayscale
image = cv2.imread('chip.png')
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Original Image
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')

# Step 3: Use Global Thresholding to segment the image
_, global_thresholderd = cv2.threshold(
    gray_image,
    127,
    255,
    cv2.THRESH_BINARY
)

# Step 4: Use Adaptive Thresholding to segment the image
adaptive_thresholderd = cv2.adaptiveThreshold(
    gray_image,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

# Step 5: Use Otsu's method to segment the image
_, otsu_thresholderd = cv2.threshold(
    gray_image,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# Global Thresholding
plt.subplot(2, 2, 2)
plt.imshow(global_thresholderd, cmap='gray')
plt.title("Global Thresholding")
plt.axis('off')

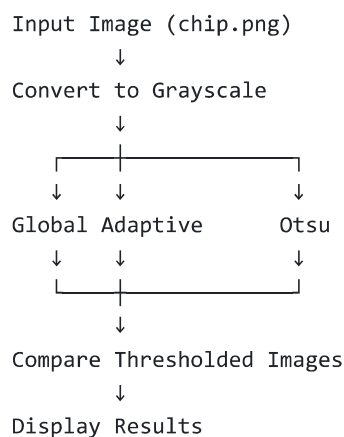
# Adaptive Thresholding
plt.subplot(2, 2, 3)
plt.imshow(adaptive_thresholderd, cmap='gray')
plt.title("Adaptive Thresholding")
plt.axis('off')

# Otsu's Method
plt.subplot(2, 2, 4)
plt.imshow(otsu_thresholderd, cmap='gray')
plt.title("Otsu's Method")
plt.axis('off')
```

```
plt.tight_layout()  
plt.show()
```

Working Principle

The complete image-processing pipeline is:



Thresholding Techniques

1. Global Thresholding

Global thresholding uses a single fixed threshold value for the complete image.

In this experiment:

Threshold = 127



It is implemented using:

```
cv2.THRESH_BINARY
```



2. Adaptive Thresholding

Adaptive thresholding determines the threshold based on local regions of the image.

The notebook uses:

```
cv2.ADAPTIVE_THRESH_GAUSSIAN_C
```



with:

```
Block Size = 11
C = 2
```



This method can be useful when different regions of an image have different illumination levels.

3. Otsu's Method

Otsu's method automatically determines a threshold value from the image.

It is implemented using:

```
cv2.THRESH_BINARY + cv2.THRESH_OTSU
```



The threshold value supplied to the function is `0` , allowing Otsu's method to calculate the threshold automatically.

Comparison

Method	Threshold Selection	Main Characteristic
Global Thresholding	Fixed value: 127	Uses one threshold for the entire image
Adaptive Thresholding	Local regions	Uses locally calculated thresholds
Otsu's Method	Automatically calculated	Automatically selects an optimal threshold

Expected Output

The final Matplotlib figure contains four outputs:

1. Original Image
2. Global Thresholding
3. Adaptive Thresholding
4. Otsu's Method

Final Output Screenshot

Paste the complete 2×2 output screenshot here

[INSERT FINAL OUTPUT IMAGE HERE]

Applications

Image thresholding is commonly used in:

- Image segmentation
- Object detection
- Document processing
- Character recognition
- Industrial inspection
- Image preprocessing
- Binary image generation

Advantages

Global Thresholding

- Simple to implement.
- Fast for processing images.
- Works well when image illumination is relatively uniform.

Adaptive Thresholding

- Handles varying illumination.
- Uses local image information.
- Useful for images with non-uniform lighting.

Otsu's Method

- Automatically selects the threshold.
- Reduces the need for manually selecting a threshold value.
- Useful for separating foreground and background in suitable images.

Limitations

Global Thresholding

- A single threshold may not work well when illumination varies across the image.

Adaptive Thresholding

- Requires selection of block size and constant values.

- Can be computationally more expensive than simple global thresholding.

Otsu's Method

- Works best when the image has suitable foreground and background intensity distributions.
- May not perform well on complex images with multiple intensity regions.

Result

Thus, the image thresholding techniques **Global Thresholding**, **Adaptive Thresholding**, and **Otsu's Method** were successfully implemented using OpenCV.

The input image `chip.png` was converted to grayscale and segmented using the three thresholding methods. The resulting images were displayed together for comparison.

Developed By

Name: Roshan V

Register No: 212225240124

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1



roshan5038